# Intro to AI with Neuroimaging Data: A end to end tutorial using PyTorch

Ninad Aithal Follow 11 min read · Nov 13, 2024

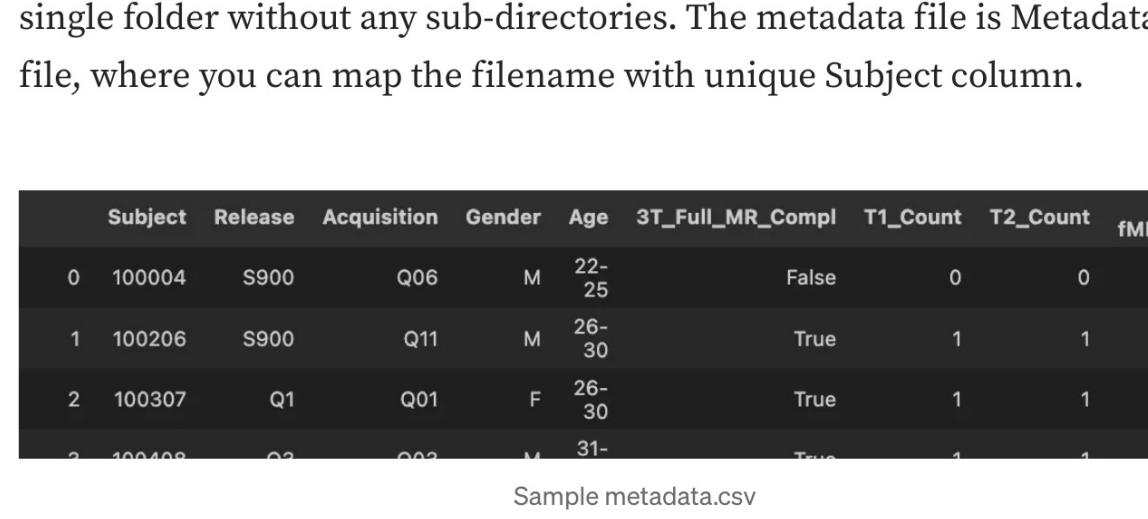
32 1

Neurolight Logo; AI generated

This is a beginner-friendly guide to using artificial intelligence with neuroimaging data. This tutorial takes you step-by-step through the entire process, starting from loading and understanding MRI data, to building and training deep learning models in PyTorch. Along the way, you'll learn essential AI and machine learning concepts, see how to set up a dataset for training, and explore real-world tasks like classifying brain images. With clear explanations and practical examples, this tutorial is designed to make complex ideas easy to understand, helping you get hands-on experience with AI in healthcare applications and giving you the skills to work on exciting neuroimaging projects from start to finish.

## File Structure and Labels

You must be able to uniquely match your file with it's corresponding entry in the metadata. For this purpose always encode a unique subject id usually provided by the dataset as a part of the filename. For the purpose of this project I have used preprocessed HCP dataset. The NifTi images are named as `Sub-100004.nii.gz` & `Sub-100206.nii.gz` etc. All the files are located in a single folder without any sub-directories. The metadata file is Metadata.csv file, where you can map the filename with unique Subject column.



| | Subject | Release | Acquisition | Gender | Age | 3T_Full_MR_Compl | T1_Count | T2_Count | 3T_RS-fMRI_Count |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 100004 | S900 | Q06 | M | 22-25 | False | 0 | 0 | 0 |
| 1 | 100206 | S900 | Q11 | M | 26-30 | True | 1 | 1 | 4 |
| 2 | 100307 | Q1 | Q01 | F | 26-30 | True | 1 | 1 | 4 |

Sample metadata.csv

### Importing necessary libraries

```
import os
import logging
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.model_selection import train_test_split
import numpy as np
import logging
import datetime
import nibabel as nib
import pandas as pd
import seaborn as sns
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
from torch.utils.data import DataLoader, Subset, random_split, Dataset
from torch.utils.data import *
```

### Hyper parameters

Initialing all the required hyper-parameters **forefront**. (Important !!, As you get experience, you'll understand why !!, For now, Take my word)

```
VAL_RATIO = 0.2
TEST_RATIO = 0.2
TRAIN_RATIO = 1 - VAL_RATIO - TEST_RATIO
DATA_PARALELL = True
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATASET = 'HCP'
PHASE = 'TRAIN'
K_FOLDS = 5

TASK = 'classification'
ROOT_DIR = 'path/to/dataset/HCP'
LABEL_DIR='path/to/metadata/metadata.csv'
LOG_DIR = 'path/to/scripts/logs'
timestamp = datetime.datetime.now().strftime('%d_%m_%Y_%H_%M_%S')
RESULT_DIR = f'path/to/scripts/results/run_{timestamp}'
os.makedirs(RESULT_DIR,exist_ok=True)
TEST_BATCH_SIZE = 2

BATCH_SIZE=16
LEARNING_RATE=0.0001
NUM_EPOCHS = 10
EARLY_STOPPING_PATIENCE = 5

NP_SEED = 42
TORCH_SEED = 36
```

### Setting up the Seed

```
np.random.seed(NP_SEED)
torch.manual_seed(TORCH_SEED)
```

### Setting up logger

```
def setup_logger(logs_dir=LOG_DIR,dataset=None,phase=None):
    os.makedirs(logs_dir, exist_ok=True)
    logger = logging.getLogger('RunLogger')
    logger.setLevel(logging.INFO)

    if not logger.hasHandlers():
        timestamp = datetime.datetime.now().strftime('%d_%m_%Y_%H_%M_%S')
        file_handler = logging.FileHandler(os.path.join(logs_dir, f'{phase}_{dat
        formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s
        file_handler.setFormatter(formatter)
        logger.addHandler(file_handler)

    return logger

LOGGER = setup_logger(logs_dir=LOG_DIR,phase=PHASE,dataset=DATASET)
```

## Dataset Class

```
class Fieldmapdata(Dataset):
    """
    Dataset class for loading neuroimaging data and associated labels for classi

    Attributes:
        root_dir (str): Path to the directory containing image data files (.nii
        label_dir (str): Path to the CSV file containing labels for each subject
        task (str): Specifies the task type: classification (Gender) or regressi
    """
    def __init__(self, root_dir, label_dir, task='classification'):
        self.labels_df = self.load_labels(label_dir) #load the labels
        self.samples = self.make_dataset(root_dir, task=task) #assign the labels

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, label = self.samples[idx]
        nifti_data = nib.load(img_path)
        data = nifti_data.get_fdata()
        image_tensor = torch.tensor(data, dtype=torch.float32).unsqueeze(0) #uns
        label_tensor = torch.tensor(label, dtype=torch.float32).unsqueeze(0)
        return image_tensor, label_tensor

    def make_dataset(self, root_dir, task = None):
        samples = []
        labels_df = self.labels_df
        for root, _, fnames in os.walk(root_dir):
            for fname in fnames:
                if fname.endswith(".nii.gz") or fname.endswith(".nii"):
                    path = os.path.join(root, fname)
                    id_ = self.extract_id_from_filename(fname)
                    try:
                        if task == 'regression':
                            label = labels_df[labels_df['Subject'] == id_]['Age'
                        if task == 'classification':
                            label = labels_df[labels_df['Subject'] == id_]['Gend
                        samples.append((path, label))
                    except:
                        continue
        return samples

    def extract_id_from_filename(self, fname):
        '''match the filename to key to query in the labels dictionary'''
        fname = fname.replace("sub-","")
        if fname.endswith("_ad.nii.gz"):
            id_ = fname.replace("_ad.nii.gz", "")
        elif fname.endswith("_rd.nii.gz"):
            id_ = fname.replace("_rd.nii.gz", "")
        elif fname.endswith("_adc.nii.gz"):
            id_ = fname.replace("_adc.nii.gz", "")
        elif fname.endswith("_fa.nii.gz"):
            id_ = fname.replace("_fa.nii.gz", "")
        elif fname.endswith(".nii.gz"):
            id_ = fname.replace(".nii.gz", "")
        return id_

    def load_labels(self, label_path):
        '''tip: use astype(required datatype)'''
        df = pd.read_csv(label_path)
        df_filtered = df[['Subject', 'Gender', 'Age']].copy()
        df_filtered['Gender'] = df_filtered['Gender'].map({'M': 0, 'F': 1}).asty
        df_filtered['Age'] = df_filtered['Age'].apply(lambda x: (int(x.split('-'
        # In HCP-Y age is a bin of 4 years, here i can assigning the average val
        df_filtered['Subject'] = df_filtered['Subject'].astype(str)
        return df_filtered
```

> *The `Fieldmaps` class is a custom PyTorch dataset designed to load neuroimaging data for classification or regression tasks, handling both image files and associated labels. It expects a directory of images (NIfTI files) and a csv file containing labels for each subject. When instantiated, the class initializes by loading the labels from the csv and preparing a list of samples, where each sample pairs an image path with its corresponding label. This process, managed by the `make_dataset` function, iterates through the directory of images, retrieves the subject ID from each filename using the `extract_id_from_filename` method, and matches it to the appropriate label from the csv file. The labels are processed in `load_labels`, where '**Gender**' is encoded numerically (0 for male and 1 for female), and '**Age**' is computed as an average for any age bins specified in ranges.*
>
> *The `__getitem__` method loads an image and its corresponding label, converting the image data to a PyTorch tensor with a channel dimension added for compatibility with neural networks. This method allows each sample to be easily accessed by an index, making the class compatible with PyTorch's DataLoader. This dataset structure provides flexibility to handle both classification (e.g., gender prediction) and regression tasks (e.g., age prediction) using the same base code, simplifying loading and preparing neuroimaging data for deep learning applications.*

### Create the dataset and split into training and testing

```
dataset = Fieldmapdata(root_dir=ROOT_DIR,label_dir=LABEL_DIR,task=TASK)
train_dataset, val_dataset, test_dataset = random_split(dataset, [TRAIN_RATIO, V
```

### Dataloader

```
train_dl = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_dl = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_dl = DataLoader(test_dataset, batch_size=TEST_BATCH_SIZE, shuffle=False)
```

### Comparing the shape of image at each stage

```
RANDOM_IMAGE_PATH = 'path/to/dataset/sub-100610.nii.gz'
raw_image = nib.load(RANDOM_IMAGE_PATH).get_fdata()
train_image = train_dataset.__getitem__(1)[0]
train_batch = next(iter(train_dl))

print('Raw image shape',raw_image.shape,'\n' 'Train image shape', train_image.sh
```

> *Raw image shape → (91, 109, 91)*
> *Train image shape → torch.Size([1, 91, 109, 91])*
> *Batch shape → torch.Size([16, 1, 91, 109, 91])*

### Stratify the labels for classification

```
def stratified_split_classification(dataset, test_size):
    labels = [dataset[i][-1] for i in range(len(dataset))]  # Assuming label is
    # cannot access sample variable for all instances, will thorw AttributeError
    train_indices, test_indices = train_test_split(
        range(dataset.__len__()), test_size=test_size, stratify=labels
    )
    train_ds = Subset(dataset, train_indices)
    test_ds = Subset(dataset, test_indices)
    return train_ds, test_ds

def splitting_data(dataset, TEST_RATIO: float, stratify: bool=False):
    if stratify:
        train_ds, test_ds = stratified_split_classification(dataset, TEST_RATIO)
    else:
        train_ds, test_ds = random_split(dataset, [1-TEST_RATIO, TEST_RATIO])
    return train_ds, test_ds
```

Get Ninad Aithal's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

```
train_and_val_ds, test_ds = stratified_split_classification(dataset, test_size=T
train_ds, val_ds = stratified_split_classification(train_and_val_ds, test_size=V
LOGGER.info('Data split completed !!')
```

```
dataset_labels = []
for ds in [train_ds, val_ds, test_ds]:
    labels = []
    for i, j in ds:
        labels.append(j)
    dataset_labels.append(labels)

for num, phase in enumerate(['Train', 'Val', 'Test']):
    Total_subjects = len(dataset_labels[num]),
    males_count = len(dataset_labels[num]) - sum(dataset_labels[num])
    females_count = sum(dataset_labels[num]),
    ratio = (len(dataset_labels[num]) - sum(dataset_labels[num])) / sum(dataset_
    print(f'Phase {phase}, total: {Total_subjects[0]}, ratio: {ratio}')
```

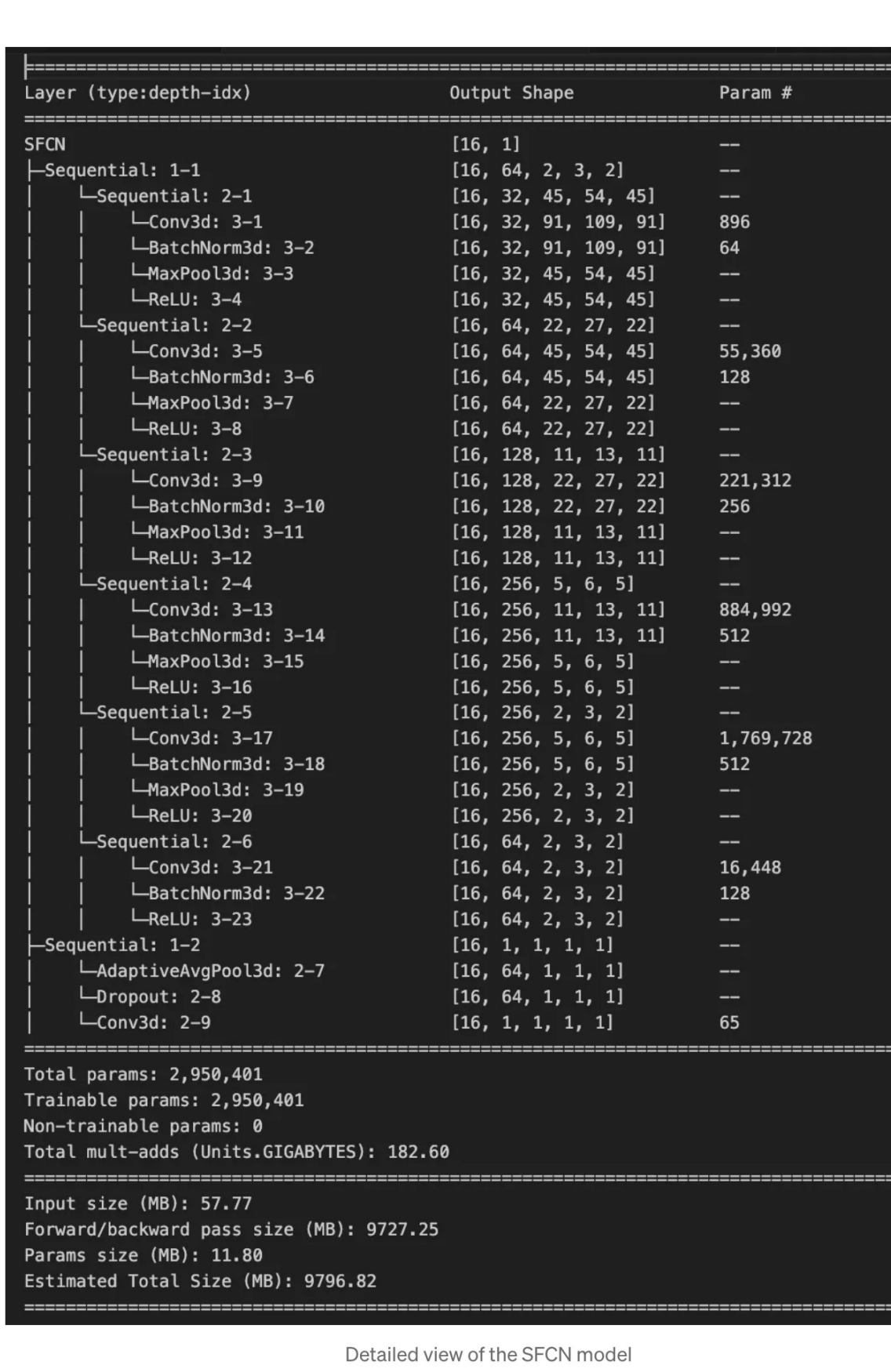
> *Phase Train, total: 542, ratio: tensor([0.8007])*
> *Phase Val, total: 136, ratio: tensor([0.7895])*
> *Phase Test, total: 170, ratio: tensor([0.7895])*
>
> *The code defines functions to split a dataset into training, validation, and test sets, with the option to stratify the splits based on class labels. `stratified_split_classification` performs a stratified split, ensuring that the distribution of labels in the train and test sets matches the original dataset's distribution, which is particularly useful in classification tasks to prevent class imbalance. The `splitting_data` function uses this stratified approach if `stratify=True`; otherwise, it performs a random split.*
>
> *After splitting, the code counts and logs the total number of subjects, along with the counts and ratio of males to females, for each phase (Train, Val, Test). Stratification is crucial when there is class imbalance, as it ensures that each split has a similar proportion of classes, helping to prevent biased model training and evaluation results.*

```
train_dl = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_dl = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_dl = DataLoader(test_dataset, batch_size=TEST_BATCH_SIZE, shuffle=False)
```

## Model

```
'''copied from https://github.com/ha-ha-ha-han/UKBiobank_deep_pretrain/blob/mast
'''This code is hardcoded for a specific input shape: ie. [batch_size, 1, 160, 1

class SFCN(nn.Module):
    def __init__(self, channel_number=[32, 64, 128, 256, 256, 64], output_dim=1,
        super(SFCN, self).__init__()
        n_layer = len(channel_number)
        self.feature_extractor = nn.Sequential()
        for i in range(n_layer):
            if i == 0:
                in_channel = 1
            else:
                in_channel = channel_number[i-1]
            out_channel = channel_number[i]
            if i < n_layer-1:
                self.feature_extractor.add_module('conv_%d' % i,
                                                  self.conv_layer(in_channel,
                                                                  out_channel,
                                                                  maxpool=True,
                                                                  kernel_size=3,
                                                                  padding=1))
            else:
                self.feature_extractor.add_module('conv_%d' % i,
                                                  self.conv_layer(in_channel,
                                                                  out_channel,
                                                                  maxpool=False,
                                                                  kernel_size=1,
                                                                  padding=0))
        self.classifier = nn.Sequential()
        ############################# Hard coded ##############################
        #avg_shape = [5, 6, 5]
        #self.classifier.add_module('average_pool', nn.AvgPool3d(avg_shape))
        ############################# Hard coded ##############################

        ############################# Change ##################################
        self.classifier.add_module('average_pool', nn.AdaptiveAvgPool3d(1))
        ############################# Change ##################################

        if dropout is True:
            self.classifier.add_module('dropout', nn.Dropout(0.5))
        i = n_layer
        in_channel = channel_number[-1]
        out_channel = output_dim
        self.classifier.add_module('conv_%d' % i,
                                   nn.Conv3d(in_channel, out_channel, padding=0,

    @staticmethod
    def conv_layer(in_channel, out_channel, maxpool=True, kernel_size=3, padding
        if maxpool is True:
            layer = nn.Sequential(
                nn.Conv3d(in_channel, out_channel, padding=padding, kernel_size=
                nn.BatchNorm3d(out_channel),
                nn.MaxPool3d(2, stride=maxpool_stride),
                nn.ReLU(),
            )
        else:
            layer = nn.Sequential(
                nn.Conv3d(in_channel, out_channel, padding=padding, kernel_size=
                nn.BatchNorm3d(out_channel),
                nn.ReLU()
            )
        return layer

    def forward_original (self, x):
        out = list()
        x_f = self.feature_extractor(x)
        x = self.classifier(x_f)
        x = F.log_softmax(x, dim=1)
        out.append(x)
        return out

    def forward(self, x):
        '''Instead of list output datatype i prefer tensor and as we are perform
        # print('In shape', x.shape)
        x_f = self.feature_extractor(x)
        # print('After feature extraction module shape', x_f.shape)
        x = self.classifier(x_f)
        # print('After classification module shape', x.shape)
        x = x.view(x.size(0), -1) #flattening
        # print('After Flattening shape', x.shape)
        return x
```

```
from torchinfo import summary
model = SFCN()
summary(model=model, input_size=(16,1,91,109,91))
```



```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
SFCN                                     [16, 1]                   --
├─Sequential: 1-1                        [16, 64, 2, 3, 2]         --
│    └─Sequential: 2-1                   [16, 32, 45, 54, 45]      --
│    │    └─Conv3d: 3-1                  [16, 32, 91, 109, 91]     896
│    │    └─BatchNorm3d: 3-2             [16, 32, 91, 109, 91]     64
│    │    └─MaxPool3d: 3-3               [16, 32, 45, 54, 45]      --
│    │    └─ReLU: 3-4                    [16, 32, 45, 54, 45]      --
│    └─Sequential: 2-2                   [16, 64, 22, 27, 22]      --
│    │    └─Conv3d: 3-5                  [16, 64, 45, 54, 45]      55,360
│    │    └─BatchNorm3d: 3-6             [16, 64, 45, 54, 45]      128
│    │    └─MaxPool3d: 3-7               [16, 64, 22, 27, 22]      --
│    │    └─ReLU: 3-8                    [16, 64, 22, 27, 22]      --
│    └─Sequential: 2-3                   [16, 128, 11, 13, 11]     --
│    │    └─Conv3d: 3-9                  [16, 128, 22, 27, 22]     221,312
│    │    └─BatchNorm3d: 3-10            [16, 128, 22, 27, 22]     256
│    │    └─MaxPool3d: 3-11              [16, 128, 11, 13, 11]     --
│    │    └─ReLU: 3-12                   [16, 128, 11, 13, 11]     --
│    └─Sequential: 2-4                   [16, 256, 5, 6, 5]        --
│    │    └─Conv3d: 3-13                 [16, 256, 11, 13, 11]     884,992
│    │    └─BatchNorm3d: 3-14            [16, 256, 11, 13, 11]     512
│    │    └─MaxPool3d: 3-15              [16, 256, 5, 6, 5]        --
│    │    └─ReLU: 3-16                   [16, 256, 5, 6, 5]        --
│    └─Sequential: 2-5                   [16, 256, 2, 3, 2]        --
│    │    └─Conv3d: 3-17                 [16, 256, 5, 6, 5]        1,769,728
│    │    └─BatchNorm3d: 3-18            [16, 256, 5, 6, 5]        512
│    │    └─MaxPool3d: 3-19              [16, 256, 2, 3, 2]        --
│    │    └─ReLU: 3-20                   [16, 256, 2, 3, 2]        --
│    └─Sequential: 2-6                   [16, 64, 2, 3, 2]         --
│    │    └─Conv3d: 3-21                 [16, 64, 2, 3, 2]         16,448
│    │    └─BatchNorm3d: 3-22            [16, 64, 2, 3, 2]         128
│    │    └─ReLU: 3-23                   [16, 64, 2, 3, 2]         --
├─Sequential: 1-2                        [16, 1, 1, 1, 1]          --
│    └─AdaptiveAvgPool3d: 2-7            [16, 64, 1, 1, 1]         --
│    └─Dropout: 2-8                      [16, 64, 1, 1, 1]         --
│    └─Conv3d: 2-9                       [16, 1, 1, 1, 1]          65
==========================================================================================
Total params: 2,950,401
Trainable params: 2,950,401
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 182.60
==========================================================================================
Input size (MB): 57.77
Forward/backward pass size (MB): 9727.25
Params size (MB): 11.80
Estimated Total Size (MB): 9796.82
==========================================================================================
```

Detailed view of the SFCN model

### Initializing Model, Optimizer & Loss

```
model = SFCN().to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
criterion = nn.BCEWithLogitsLoss()
# criterion2 = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(1.28,dtype=torch.flo
# Females are class 1, ratio of class to class 0 --> 504/395 = 1.27
```

### Simple Training and Validation loop

```
train_losses, val_losses = [], []
train_accuracies, val_accuracies = [], []

for epoch in range(NUM_EPOCHS):
    model.train()
    train_loss = 0.0
    train_total = 0
    train_correct = 0

    for inputs, labels in train_dl:
        inputs = inputs.to(DEVICE)
        labels = labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(inputs)
        tloss = criterion(outputs, labels)
        tloss.backward()
        optimizer.step()
        train_loss += tloss.item() / inputs.size(0)
        probability = torch.sigmoid(outputs) # during inference you have to appl
        predicted = (probability >= 0.5).float()
        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()

    train_accuracy = 100 * train_correct / train_total
    train_accuracies.append(train_accuracy)
    train_losses.append(train_loss)

    model.eval()
    val_loss = 0.0
    val_total = 0
    val_correct = 0

    with torch.no_grad():
        for inputs, labels in val_dl:
            inputs = inputs.to(DEVICE)
            labels = labels.to(DEVICE)
            outputs = model(inputs)
            vloss = criterion(outputs, labels)
            val_loss += vloss.item() / inputs.size(0)
            probability = torch.sigmoid(outputs)
            predicted = (probability >= 0.5).float()
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

        val_accuracy = 100 * val_correct / val_total
        val_accuracies.append(val_accuracy)
        val_losses.append(val_loss)

        print(f"Epoch [{epoch+1}/{NUM_EPOCHS}]: Train Loss: {train_loss:.4f} Tra
        LOGGER.info(f"Epoch [{epoch+1}/{NUM_EPOCHS}]: Train Loss: {train_loss:.4
torch.save(model.state_dict(),'best_model.pt')
```

> *Epoch [1/10]: Train Loss: 1.2264 Train Accu: 66.01% Val Loss: 1.3759 Val Accu: 56.47%*
> *Epoch [2/10]: Train Loss: 0.9750 Train Accu: 80.94% Val Loss: 2.1438 Val Accu: 56.47%*

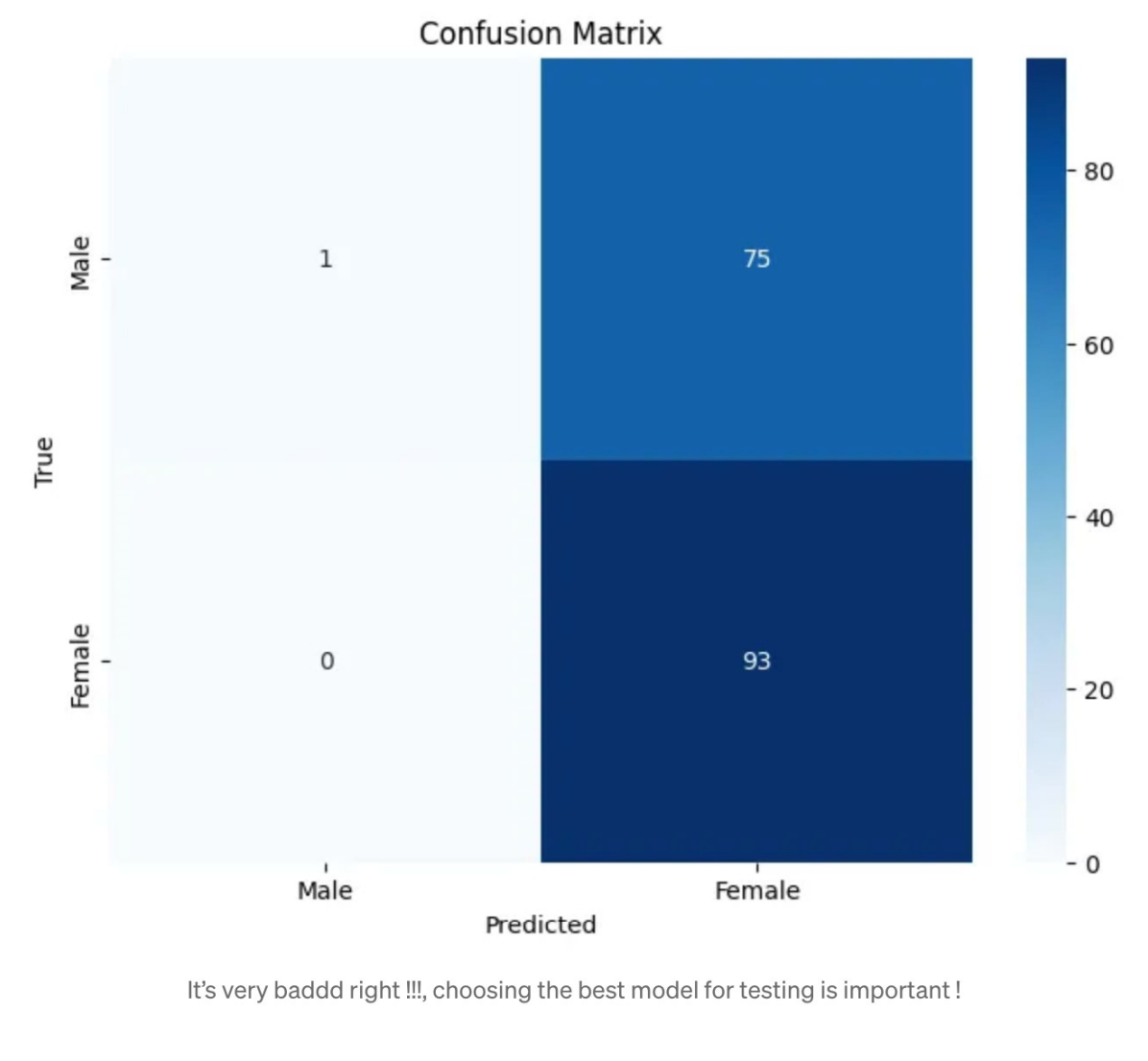
### Save the Best model, performing the best on Validation dataset

```
best_model = SFCN()
# best_model.load_state_dict('best_model.pt')

#using the above model for now
new_state_dict = {}
for key, value in model.state_dict().items():
    new_key = key.replace("module.", "")
    new_state_dict[new_key] = value
best_model.load_state_dict(new_state_dict)
```

### Test Predictions

```
best_model.to(DEVICE)
best_model.eval()
test_loss = 0.0
test_total = 0
test_correct = 0

with torch.no_grad():
    all_predicted = []
    all_labels = []
    for inputs, labels in test_dl:
        inputs = inputs.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = best_model(inputs)
        te_loss = criterion(outputs, labels)
        test_loss += te_loss.item()/inputs.size(0)
        probability = torch.sigmoid(outputs)
        predicted = (probability >= 0.5).float()
        all_predicted.extend(predicted.cpu().numpy().ravel().tolist())
        all_labels.extend(labels.cpu().numpy().ravel().tolist())
        test_total += labels.size(0)
        test_correct += (predicted == labels).sum().item()

    test_accuracy = 100 * test_correct / test_total

print(f"TEST LOSS: {test_loss}, TEST ACCURACY: {test_accuracy}")
LOGGER.info(f"TEST LOSS: {test_loss}, TEST ACCURACY: {test_accuracy}")
accuracy = accuracy_score(all_labels, all_predicted)
sensitivity = recall_score(all_labels, all_predicted)
print(f'Accuracy: {accuracy}, Sensitivity (Recall): {sensitivity}')
LOGGER.info(f'Accuracy: {accuracy}, Sensitivity (Recall): {sensitivity}')

cm = confusion_matrix(all_labels, all_predicted)
print('CM: ',cm)
LOGGER.info(f"CM : {cm}")
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",xticklabels=['Male', 'Female']
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.savefig('cm.png')
```

> *TEST LOSS: 42.06333679519594, TEST ACCURACY: 55.62130177514793*
> *Accuracy: 0.5562130177514792,*
> *Sensitivity (Recall): 1.0*



It's very baddd right !!!, choosing the best model for testing is important !

### K-Fold Cross Validation for Training with early stopping

```
kf = KFold(n_splits=K_FOLDS, shuffle=True, random_state=NP_SEED)
fold_train_losses, fold_val_losses = [], []
fold_train_accuracies, fold_val_accuracies = [], []
LOGGER.info(f"{K_FOLDS} FOLDS CROSS VALIATION TARINING STARTED")

for fold, (train_idx, val_idx) in enumerate(kf.split(train_dataset)):
    print(f'Fold {fold+1}/{K_FOLDS}')
    LOGGER.info(f'Fold {fold+1}/{K_FOLDS}')

    train_ds = Subset(train_dataset, train_idx)
    val_ds = Subset(train_dataset, val_idx)

    train_dl = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
    val_dl = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False)

    model = SFCN().to(DEVICE)
    model = nn.DataParallel(model)
    optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

    criterion = nn.BCEWithLogitsLoss()

    best_val_loss = np.inf
    epochs_without_improvement = 0

    # Train the model for each fold
    for epoch in range(NUM_EPOCHS):
        model.train()
        train_loss = 0.0
        train_total = 0
        train_correct = 0

        for inputs, labels in train_dl:
            inputs = inputs.to(DEVICE)
            labels = labels.to(DEVICE)
            optimizer.zero_grad()
            outputs = model(inputs)
            tloss = criterion(outputs, labels)
            tloss.backward()
            optimizer.step()
            train_loss += tloss.item() / inputs.size(0)
            probability = torch.sigmoid(outputs)
            predicted = (probability >= 0.5).float()
            train_total += labels.size(0)
            train_correct += (predicted == labels).sum().item()
```